Civic Issue Reporter

A Campus Infrastructure Issue Reporting and Resolution Platform
Project Proposal for UCS503P
Submitted to: Ms. Anushka
Thapar Institute of Engineering and Technology

Lavanya Sharma, COE* Parismita Thapa, COE† Rudra Yadav, COE‡
Nadeem Esrar, COE§

August 18, 2026

1 Higher-Order Goal

This project applies standard software engineering practice, including requirement analysis, system design, and structured development, to a real campus problem. The broader motivation is to improve campus infrastructure governance by reducing the time and effort required to resolve civic issues reported by students and faculty. The immediate engineering objective is to design and implement a measurable, deployable software solution for efficient, department-routed issue resolution across the Thapar campus, including academic blocks, hostels, the library, cafeterias, the sports complex, and other common areas.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core reporting-and-tracking workflow and validating it with a small group of students and hostel residents within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations, deferring features like auto-escalation and notifications until the core flow is stable, and targeting CI-backed automation early.
- Success is defined by what can be delivered and measured in the first iteration, a working submission-to-status flow, then improved based on user feedback before adding duplicate detection and escalation.

3 Problem Statement

Campus infrastructure faults - broken equipment, plumbing leaks, faulty wiring, damaged furniture, non-functional washrooms, unsafe walkways - are today reported informally: through hostel WhatsApp groups, verbal complaints to wardens, or ad-hoc emails to department staff. This produces several concrete problems:

- **No single source of truth:** the same issue is reported independently to multiple people, with no shared record of what has already been flagged or fixed.

*Roll No: [1024030227], Email: lsharma3_be24@thapar.edu
†Roll No: [1024030309], Email: pthapa_be24@thapar.edu
‡Roll No: [1024030389], Email: ryadav_be24@thapar.edu
§Roll No: [1024030291], Email: nesrar_be24@thapar.edu

- **No accountability or tracking:** a reporter has no way to check whether their complaint was even received, let alone what stage it is at.
- **No severity-aware prioritization:** a loose wire in a washroom and a broken chair in a common room compete for the same informal attention, with no structural way to surface safety-critical issues first.
- **Redundant effort:** without a shared, searchable list of open issues, students and faculty re-report problems that are already known, and administrators re-investigate what has already been logged.
- **No institutional visibility:** there is no aggregate view - by building, department, or category - that lets facilities management identify recurring or high-impact problem areas.

Why this matters now: TIET's campus spans multiple hostels, academic blocks, and shared facilities maintained by different departments. As the reporting channel is informal and department-specific, the effective bottleneck is not maintenance capacity but *issue visibility and routing*. A structured reporting and tracking system directly addresses that bottleneck without requiring any change to how maintenance work itself is carried out.

4 Objectives

Primary goal: To design and deploy a role-based web platform that gives the TIET campus community a structured, trackable channel for reporting and resolving infrastructure issues, while reducing duplicate reporting and improving response time to high-priority faults.

- Give students and faculty a single, low-friction channel to report infrastructure issues with a photo, category, and precise location.
- Automatically route each report to the right department, while still allowing a manual override when the auto-suggested category is wrong.
- Prevent duplicate reports of the same problem from fragmenting attention, by detecting likely duplicates at submission time and letting users upvote an existing report instead.
- Ensure high-impact or safety-critical issues are not left to languish in a queue by escalating priority automatically once an issue crosses a configurable upvote threshold.
- Close the feedback loop for reporters through status-change and escalation notifications, so that "I reported it and heard nothing" is no longer the default experience.
- Give department administrators and a Super-Admin department-scoped and institution-wide visibility respectively, so ownership of an issue is always clear.

5 Proposed Solution

5.1 User Roles

Access is role-based, with two reporting roles and two administrative roles:

Role	Report	View all	Upvote	Change status	Scope
Student	✓	✓	✓	–	–
Faculty	✓	✓	✓	–	–
Administrator	–	✓	–	✓	Own department
Super-Admin	–	✓	–	✓	All departments

Students and faculty are treated identically: both can submit reports, browse the full (read-only) list of reported issues, and upvote an existing report. Administrators are scoped to a single

department/category and move issues through their lifecycle. The Super-Admin has full cross-department visibility and control, and is additionally the only role notified on every escalation event regardless of which department owns the issue; this gives an institutional oversight layer even when an owning Administrator has not yet acted.

5.2 Core Features

- **Structured reporting:** photo upload, category selection (with automatic department suggestion, manually overridable by the reporter or an admin), and location entry via a *Building* → *Floor* → *Area/Room* hierarchy (dropdowns plus free text), rather than GPS or a map pin.
- **Duplicate detection at submission time:** the system checks the new report against *open* issues in the same location/department bucket, using location match, category match, and NLP-based description similarity. If a likely duplicate is found, the reporter is shown it and prompted to upvote instead of filing a new report.
- **Auto-escalation:** when an open issue's upvote count crosses a configurable, per-category threshold, it is flagged *High Priority*, surfaced at the top of the owning Administrator's and the Super-Admin's dashboards, and triggers a Super-Admin notification. Escalation is purely a priority/visibility flag; it does not change the underlying status lifecycle.
- **Status tracking:** every issue moves through `Reported` → `Ongoing` → `Finished`, updated only by the Administrator of the owning department or the Super-Admin.
- **Notifications:** in-app notifications for status changes (to the original reporter), upvote/duplicate activity (to the original reporter, so they know their issue is gaining traction), new reports landing in a department (to that department's Administrator), and escalations (to the Super-Admin).

5.3 Core Workflow

1. A student or faculty member submits an issue with a photo, category, and structured location.
2. The system auto-suggests a department and simultaneously checks for likely duplicates among open issues in that location/department bucket.
3. If a likely duplicate exists, the reporter is shown it and can upvote it instead of creating a new record; if not, a new issue is created in `Reported` status and routed to the relevant Administrator, who is notified.
4. As upvotes accumulate, the system checks the issue's count against its category's escalation threshold on every upvote event; on crossing it, the issue is marked *High Priority* and the Super-Admin is notified.
5. The owning Administrator (or Super-Admin) moves the issue through `Reported` → `Ongoing` → `Finished`; the original reporter is notified at each transition.

5.4 Operational Constraints for an Educational, Tier-2/3 Setting

- Keep the solution usable on low-to-mid range student devices and typical hostel/campus network conditions, via responsive web design rather than a heavier native app.
- Avoid dependence on research-grade algorithms for duplicate detection or routing; prioritize workflow correctness, routing accuracy, and system reliability over algorithmic novelty.
- Design for deployability using common managed hosting and a managed database, since the team does not have access to dedicated operations infrastructure within the course timeline.

6 Solution Approach

This is an engineering course project, so the emphasis is on scalable workflow and system-engineering concerns, duplicate handling, service architecture, and delivery pipeline, rather than on novel algorithm research.

6.1 Duplicate/Quality Assistance

Duplicate detection is heuristic-based, combining three signals so that no single weak signal can produce a false match on its own:

1. **Location match** - exact match on building and floor, fuzzy match on the free-text area/room field.
2. **Department/category match** - the new report is only compared against issues already routed to the same department.
3. **Description similarity** - a simple similarity measure (e.g., token overlap or TF-IDF cosine similarity with a threshold, without any claim to state-of-the-art accuracy) between the new description and existing open descriptions within the matched location/department bucket.

The comparison set is restricted to issues still open (not `Finished`), on the reasoning that a resolved issue recurring is a *new* problem worth a fresh report, not a duplicate. The system surfaces the top candidate to the reporter for a human decision, and it never fully auto-merges, auto-deletes, or auto-rejects a submission and keeping a human in the loop for a step that directly affects data quality.

6.2 Web Architecture

A typical 3-tier web architecture:

- **Frontend:** React, a responsive UI for both reporters (submission form, status view) and administrators (dashboard), usable on low-to-mid range student devices.
- **Backend API:** Node.js/Express (or Flask as a fallback), exposing a REST API for authentication, submission, routing, status transitions, and escalation logic.
- **Database:** PostgreSQL, chosen for relational integrity between issues, departments, users, upvotes, and notifications, and for straightforward indexing on the fields duplicate detection and dashboards query most (location, department, status).

Supporting and Infrastructure: Cloudinary or AWS S3 for image attachments, kept out of the primary database; JWT-based sessions with role-based access control enforced at the API layer for all four roles; and notification delivery via polling rather than WebSockets, to keep the MVP's infrastructure and failure modes simple.

6.3 Data Model Highlights

- An `issues` table carries category, department, structured location fields, status, an upvote count, and a `priority` field (`Normal`/`High`) set by the escalation check.
- An `escalation_threshold` value, configurable by the Super-Admin and tunable per category, is checked against the upvote count on every upvote event.
- A `notifications` table (`user_id`, `issue_id`, `type`, `message`, `read_status`, `timestamp`) backs all four notification types described above.
- Duplicate-location matching is a structured field comparison (exact building/floor, fuzzy area/room text) rather than a geospatial radius calculation which is a deliberate trade-off, since GPS is unreliable indoors and map APIs add cost and complexity the MVP does not need.

6.4 CI/CD and Fast Delivery Enabler

CI/CD will be used to:

- Automatically build and run tests (backend unit tests, linting) on every change.
- Produce repeatable, versioned deployment artifacts to a staging environment.
- Enable safe, frequent releases of incremental features, in line with the weekly iteration cadence described under Time-to-value.

7 Auto-Escalation Design

A single flat upvote threshold across every category would either (a) be too high for safety-critical issues, which should not have to wait for popularity to earn attention, or (b) be too low for high-traffic areas, where "High Priority" would stop meaning anything. The design instead makes the threshold configurable per category, set by the Super-Admin:

- **Safety-critical** (electrical, structural, fire safety): a low threshold (∼2-3 upvotes) - severity, not volume, should drive escalation here.
- **High-traffic common areas** (library, cafeteria, sports complex): a higher threshold (∼8-12 upvotes), since large natural visibility means genuine problems accumulate upvotes quickly on their own.
- **Hostel/block-specific issues:** an intermediate threshold (∼5-8 upvotes), reflecting a smaller audience per location.
- **Low-severity/cosmetic issues** (paint, furniture): a higher bar (∼15+ upvotes), so that "High Priority" is not diluted.

For the MVP, a single **default threshold of 5 upvotes** is used across all categories, since no real usage data from TIET exists yet to calibrate per-category values responsibly. This default is treated as a known, temporary simplification, explicitly intended to be reviewed and tuned per category after the pilot, once real reporting-volume data is available, not as a final design decision.

8 Evaluation Criteria

The system's original README does not specify success metrics; the following are proposed based on the features actually being built, so that each stated feature has a corresponding measurable outcome.

8.1 Primary Metrics

- **Time-to-first-action (TTFA):** time between an issue entering `Reported` and its first transition to `Ongoing`, measured from database timestamps. Target: median TTFA within a defined pilot-window bound (to be set from pilot baseline data, since no institutional baseline currently exists).
- **Duplicate-detection precision:** of the duplicate candidates surfaced to reporters, the proportion that reporters agree are genuine duplicates (accepted upvote vs. rejected and filed as new). This is directly measurable from user choices at submission time and avoids needing a hand-labelled test set.

8.2 Secondary Metrics

- **Escalation precision:** proportion of auto-escalated issues that Administrators, on review, agree warranted High Priority: a check on whether the MVP's flat threshold is behaving

reasonably ahead of per-category tuning.

- **Resolution time by category:** time from `Reported` to `Finished`, broken down by category, to identify systemic bottlenecks by department rather than only reporting an institution-wide average.
- **Duplicate deflection rate:** proportion of submissions that end as an upvote on an existing issue rather than a new report : a direct measure of how much redundant reporting is being prevented.
- **Notification effectiveness:** proportion of status-change notifications that are read (via `read_status`) within a defined window, as a proxy for whether the feedback loop is actually closing the trust gap identified in the problem statement.

8.3 Pilot Validation Plan

- Run the pilot within a bounded scope - e.g., one or two hostel blocks plus one academic block - rather than campus-wide, to keep the reporting volume interpretable within the course timeline.
- Recruit a small set of student/faculty reporters and, for Administrator accounts, either real departmental staff (if available) or team-simulated department roles.
- Collect an informal baseline (e.g., via a short survey on how long informally-reported issues currently take to get attention) to give the primary/secondary metrics a comparison point, since no existing institutional baseline is being tracked today.

9 Scalability and Deployment

1. **Theoretical scalability:**
   - Decoupled frontend/backend, communicating over a stateless REST API, so either layer can scale independently.
   - Indexing on the fields most queried by duplicate detection and dashboards (location, department, status), plus pagination on list views, so query cost does not grow linearly with total issue volume as adoption increases beyond the pilot.
   - Escalation and duplicate-detection checks scoped to a bounded bucket (same department/location, open issues only) rather than the full issue table, so their cost does not grow with total historical volume either.
2. **Practical deployability:**
   - Common managed hosting for the backend and a managed PostgreSQL instance, avoiding bespoke infrastructure.
   - Object storage (Cloudinary/S3) for images from day one, so image volume growth does not pressure the primary database.
   - CI/CD pipeline for staging/production, as detailed under CI/CD and Fast Delivery Enabler.

10 Engine Availability Heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project, so the team is not building infrastructure from scratch:

- Web framework for REST APIs (Express/Flask) and a responsive frontend framework (React) for templated/reactive pages.
- A managed relational database (PostgreSQL).

- Token-based authentication (JWT) with role-based access control.
- Object storage for photo attachments (Cloudinary/AWS S3, with local storage as a pilot fallback).
- A test framework for backend unit/integration tests.
- A CI runner (e.g., GitHub Actions) and a staging deployment target.

We will use these mature, off-the-shelf components to focus engineering effort on workflow design, duplicate/escalation logic, and measurement, rather than on inventing new infrastructure.

11 Timeline

The schedule below breaks delivery into weekly phases, in line with the iteration cadence described under Time-to-value, and includes an explicit buffer week to absorb slippage rather than letting it erode the final wrap-up:

Wk	Phase	Task	Deliverable
1–2	Planning	Requirements; schema, roles, API contract	Proposal + ER diagram
3–4	Core build	Auth (JWT/RBAC); submission form	Submission-to-DB flow
5–6	Core build	Duplicate-detection heuristic; auto-routing	Duplicate-check prototype
7–8	Core build	Admin/Super-Admin dashboards; status lifecycle	Role dashboards (MVP core)
9	Integration	Escalation logic; notifications	Escalation + notifications live
10	Testing	CI/CD pipeline; QA testing	Staging deployment, tests
11	Pilot	Rollout: 1-2 hostels + 1 academic block	Pilot data collection begins
12	**Buffer**	Bug fixes; unresolved scope	Stabilized build
13–14	Wrap-up	Metric analysis (§8.1); report prep	Final report + presentation

12 Resources and Budget

Human resources: A 4-member team dividing work across frontend (React UI for reporters and dashboards), backend/API (authentication, routing, status and escalation logic), duplicate-detection and data modelling (heuristic design, PostgreSQL schema), and DevOps (CI/CD, staging deployment, pilot coordination), coordinated through a shared Git repository.

Material resources: GitHub for version control and CI (GitHub Actions); a managed PostgreSQL instance (free/student tier, e.g., Supabase or Railway); Cloudinary/AWS S3 free tier for image storage; and free-tier hosting (e.g., Vercel/Render) for staging and pilot deployment, consistent with the Engine Availability Heuristic above.

Budget: No external funding required; all resources are available through free or managed-tier services and university- provided infrastructure.

13 Project Scope and Deliverables

13.1 MVP Deliverables

- Citizen-side (student/faculty) issue submission form: photo, category with auto-suggested department, structured location entry.
- Duplicate-candidate check at submission time (location + department + description similarity), with an upvote path.

- Department-scoped Administrator dashboard: view assigned issues, update status through `Reported` → `Ongoing` → `Finished`.
- Super-Admin dashboard: cross-department visibility, high-priority issues surfaced at the top.
- Auto-escalation with the flat MVP default threshold (5 upvotes).
- In-app notifications for all four event types (status change, upvote/duplicate, new-report-to-admin, escalation-to-Super-Admin), via polling.
- JWT-based auth with role-based access control for all four roles.
- CI: build, backend unit tests, linting on every change; CD to a staging environment.

13.2 Post-MVP / Subsequent Deliverables

- Per-category configurable escalation thresholds (replacing the flat MVP default), calibrated using real pilot upvote-volume data.
- Improved duplicate-detection similarity (e.g., embedding-based similarity instead of pure token overlap/TF-IDF) once enough real description data exists to evaluate it meaningfully.
- Aggregate analytics for facilities management: recurring-issue hotspots by building/category, resolution-time trends over time.
- Optional email/SMS notification channel alongside in-app notifications.
- Search/filtering improvements on the public issue list as report volume grows.

14 Risks and Mitigations

- **Duplicate false positives/negatives:** an overly aggressive similarity threshold could push genuinely distinct issues together, discouraging legitimate reports; an overly conservative one would fail to deflect true duplicates. *Mitigation:* always show the candidate for a human decision rather than auto-merging, and treat the duplicate-detection precision metric (Section 8.1) as a live signal for tuning the threshold.
- **Upvote gaming:** without a one-vote-per-user-per-issue constraint, upvote counts (and therefore escalation) could be manipulated. *Mitigation:* enforce a unique (`user_id`, `issue_id`) constraint at the database level on upvotes.
- **MVP threshold miscalibration:** the flat default-5 threshold is known to be a simplification and may over- or under-escalate relative to a category's real severity/volume profile. *Mitigation:* explicitly scoped as an MVP limitation (Section 7), with per-category tuning as a defined post-pilot deliverable rather than an afterthought.
- **Low pilot adoption:** a reporting tool is only useful if people use it instead of the informal channels it is meant to replace. *Mitigation:* keep the submission flow to a minimal number of fields, and scope the pilot narrowly (Section 8.3) so the team can actively drive adoption within a small, trackable population.
- **Administrator adoption:** if department staff do not update statuses, the notification loop and TTFA metric both break down. *Mitigation:* keep the Administrator update action to a single dropdown change, and simulate Administrator roles within the team if real departmental staff are unavailable during the pilot window.
- **Image storage cost/volume:** unrestricted photo uploads could grow storage cost quickly. *Mitigation:* client-side compression before upload and a per-report image-count/size cap.

15 Summary

This project proposes a civic issue reporter, a role-based web platform for reporting, routing, and resolving campus infrastructure issues at TIET. Its core engineering contributions are a submission flow that actively suppresses duplicate reports rather than merely listing them, and a priority-escalation mechanism designed from the outset to be severity-aware rather than purely volume-driven, even though the MVP ships with a single calibratable default. The proposal defines primary and secondary metrics directly tied to these mechanisms (time-to-first-action, duplicate-detection precision, escalation precision) so that the pilot produces the data needed to move from the MVP's flat defaults to the tuned, per-category design that motivated the system in the first place.